

```
<div class="logo-container">

</div>
```

```
<div class="center-title">Buku Panduan Teknis & Arsitektur Kode</div>
<div class="center-subtitle">(Dokumentasi Sistem Camelio Finance)</div>
```

Dokumen ini menjelaskan secara komprehensif mengenai arsitektur perangkat lunak, tumpukan teknologi (tech stack), alur logika sistem, skema database, dan integrasi Artificial Intelligence (Google Gemini) pada aplikasi **Camelio Finance**.

1. Tumpukan Teknologi (Tech Stack)

Aplikasi ini dibangun menggunakan teknologi modern yang berfokus pada skalabilitas dan integrasi AI:

- **Frontend / Mobile Framework:** Flutter (Dart) - Mendukung Cross-Platform.
- **State Management:** provider - Untuk mengelola state UI secara reaktif dan efisien.
- **Dependency Injection:** get_it - Mengelola *Service Locator* untuk instance service & API agar tidak membebani memori.
- **Backend & Autentikasi:** Firebase (Firebase Auth, Cloud Firestore, Firebase Remote Config).
- **Artificial Intelligence:** Google Generative AI (Gemini 1.5 Flash) untuk pemrosesan teks dan gambar.
- **Media Optimization:** Cloudinary (Mengompresi dan mengunggah gambar struk sebelum diproses oleh AI).
- **Local Storage:** shared_preferences untuk menyimpan preferensi pengguna (seperti tema atau sesi).

2. Arsitektur dan Struktur Folder (Directory Structure)

Aplikasi ini menggunakan pola arsitektur **Feature-First** (MVVM Pattern) yang memisahkan logika dan tampilan dengan tegas:

core/ (Pusat Konfigurasi & Desain)

Berisi aturan global yang tidak berubah-ubah.

- constants/: Menyimpan skema warna (Hex) dan API Key.
- theme/: Mengatur font (Outfit), ukuran teks, dan gaya komponen untuk Light & Dark Mode.
- utils/: Fungsi formatter uang, tanggal, dan utility screenshot.
- locator.dart: Menggunakan GetIt untuk mendaftarkan Service (Service Locator Pattern).

data/ (Lapisan Data & Services)

Bagian backend logic aplikasi yang berurusan dengan API.

- models/: Blueprint data (transaction_model.dart, user_model.dart, e_wallet_model.dart).
- services/:
 - auth_service.dart: Logika Firebase Auth.
 - firestore_service.dart: Komunikasi langsung dengan database Cloud Firestore.

- `gemini_ai_service.dart`: Integrasi AI Gemini.
- `cloudinary_service.dart`: Integrasi unggahan ke Cloudinary.

providers/ (Lapisan Logika Bisnis)

Mengatur aliran data agar tampilan (UI) otomatis terupdate saat data berubah.

- `auth_provider.dart`: Melacak status login.
- `transaction_provider.dart`: Pusat data keuangan (menghitung saldo total dan memegang daftar transaksi).
- `ai_insight_provider.dart`: Mengelola percakapan Chat AI.

ui/ (Antarmuka Pengguna)

Semua file yang bertanggung jawab atas tampilan visual, dipisah menjadi `pages/` (Layar Halaman Utama) dan `widgets/` (Komponen yang digunakan ulang).

3. Struktur Database (Cloud Firestore)

Sistem menggunakan arsitektur data **Flat (Koleksi Root)** untuk memudahkan *query* berkinerja tinggi (NoSQL):

1. **Koleksi: users** (Profil Pengguna)
 - Field: `balance`, `monthlyBudget`, `name`, `email`, `status`.
 2. **Koleksi: wallets** (Dompet Digital / Bank)
 - Field: `accountNumber`, `balance`, `name`, `type`, `userId`.
 3. **Koleksi: transactions** (Riwayat Transaksi)
 - Field: `amount`, `category`, `date`, `description`, `type` (income/expense), `items` (List detail belanja), `userId`.
 4. **Koleksi: notifications & rating** (Log sistem dan Feedback).
-

4. Alur Kerja Integrasi AI (Google Gemini 1.5 Flash)

Kekuatan utama aplikasi ini terletak pada integrasi Multimodal AI. Berikut alurnya:

A. Fitur Scan Struk (OCR + Vision AI)

1. Pengguna membuka kamera di dalam aplikasi dan memotret fisik struk belanja.
2. Gambar dikompresi dan dikirim ke server **Cloudinary**. Cloudinary mengembalikan URL gambar.
3. Aplikasi mengirimkan URL gambar beserta *prompt* rahasia ke **Gemini Vision AI**.
4. Gemini bertindak sebagai OCR cerdas; membaca teks, dan membedakan nominal harga dan nama barang.
5. Gemini mengembalikan hasil ekstraksi tersebut dalam format data **JSON terstruktur**.
6. Aplikasi mengubah JSON tersebut menjadi objek `TransactionModel` lalu menampilkannya sebelum dikonfirmasi pengguna untuk disimpan ke Firestore.

B. Fitur Input Suara & Chat AI (Natural Language Input)

1. Pengguna mengetik atau berbicara (contoh: "Saya beli bahan baku seharga 50 ribu").
2. Teks tersebut dikirim ke **Gemini LLM** beserta *prompt* klasifikasi khusus.
3. AI membedah makna kalimat, mendeteksi nominal angka ("50 ribu" -> 50000), dan menentukan kategori ("Bahan Baku").
4. Data dikembalikan dalam format JSON dan langsung disimpan ke Firestore.

C. AI Insight (Rekomendasi Pembelian Ekonomis)

1. Aplikasi menarik data seluruh pengeluaran (30 hari terakhir) dari Firestore.
2. Data dirangkum dan dikirim ke **Gemini LLM** dengan *prompt* khusus untuk melakukan "Analisis Keuangan UMKM".
3. AI mencari anomali pengeluaran dan merumuskan saran penghematan atau strategi pembelian bahan baku. Teks rekomendasi ditampilkan di Halaman "AI Insight".

5. Keamanan Aplikasi (Security)

1. **Proteksi API Key:** Menggunakan package flutter_dotenv (.env) dipadukan dengan Firebase Remote Config agar API Key tidak masuk dalam *source code* secara *hardcoded*.
2. **Firestore Rules:** Konfigurasi *Security Rules* Firestore memastikan pengguna hanya dapat membaca dan menulis dokumen data dengan *userId* yang cocok dengan sesi otentikasi (Auth) mereka saat ini.

Dokumen ini dihasilkan secara otomatis dari analisis arsitektur Camelio Finance.

1. Arsitektur Pattern (MVVM & Service Locator)

Aplikasi Camelio Finance dirancang menggunakan kombinasi **Model-View-ViewModel (MVVM)** melalui package provider dan **Service Locator Pattern** menggunakan `get_it`.

- **Model:** Berada di folder `lib/data/models/`. Memetakan data dari Firestore JSON ke objek Dart.
- **View:** Berada di folder `lib/ui/pages/` dan `lib/ui/widgets/`. Murni berisi kode Flutter (UI/UX) dan pasif menunggu data dari Provider.
- **ViewModel:** Berada di folder `lib/providers/`. Bertugas sebagai jembatan yang menghubungkan View dan Data/Service.
- **Service:** Berada di folder `lib/data/services/`. Menjalankan fungsi berat (API Call, eksekusi Database) yang kemudian dipanggil oleh Provider melalui `locator()`.

2. Bedah Lapisan Data (Data Layer)

Lapisan data bertanggung jawab untuk mengambil, menyimpan, dan memetakan data.

A. Data Models (`lib/data/models/`)

Setiap entitas utama di aplikasi memiliki representasi kelas Model agar *type-safe*.

1. **`user_model.dart`:** Blueprint profil user. Terdiri dari field dasar (`uid`, `name`, `email`, `monthlyBudget`,

- balance). Dilengkapi metode `fromJson` dan `toJson` untuk konversi data Firestore.
2. **transaction_model.dart**: Blueprint pencatatan kas. Menampung data nominal, kategori, catatan, serta List/Array detail barang belanjaan yang diekstrak oleh AI.
 3. **e_wallet_model.dart**: Menampung data integrasi e-wallet, termasuk ikon dan status sinkronisasi.
 4. **expense_model.dart**: Struktur data khusus untuk chart/grafik analitik keuangan.
 5. **notification_model.dart**: Data log peringatan sistem dan peringatan AI Advisor.

B. Data Services (lib/data/services/)

Service didaftarkan ke `get_it` secara *lazy singleton* sehingga hanya diinisialisasi saat dibutuhkan.

1. **gemini_ai_service.dart (Core AI)**: Modul terbesar (10KB+) yang menangani logika *prompt engineering*. Terdapat fungsi `extractReceipt()` untuk mengirim URL Cloudinary ke Gemini Vision, dan `analyzeFinancials()` untuk AI Insight.
2. **firestore_service.dart**: Pembungkus (*wrapper*) untuk semua operasi CRUD (Create, Read, Update, Delete) ke Cloud Firestore. Memastikan data dikirim sesuai Model yang benar.
3. **auth_service.dart**: Logika Firebase Auth. Meliputi Login Email/Pass, Register, dan integrasi Google Sign-In (OAuth).
4. **gmail_service.dart**: Mengakses Gmail API via OAuth 2.0 untuk memindai email mutasi masuk dari bank atau dompet digital dan menyinkronkannya sebagai data transaksi.
5. **cloudinary_service.dart**: Menggunakan REST API untuk mengunggah (POST) foto struk ke server Cloudinary sebelum foto diproses AI agar ukuran data jauh lebih kecil dan cepat.
6. **csv_logger_service.dart**: Menyediakan fitur ekspor data transaksi ke format CSV untuk diunduh pengguna.
7. **notification_service.dart**: Menginisialisasi *Flutter Local Notifications Plugin* untuk push-notification di HP pengguna.

3. Bedah Lapisan Logika Bisnis (Providers Layer)

Provider memegang *state* aplikasi. File-file ini berada di `lib/providers/` dan di-*inject* di `main.dart` menggunakan `MultiProvider`.

1. **auth_provider.dart**: Memegang status login (`isLoading`, `isAuthenticated`, `currentUser`). Menangani aliran layar dari Splash Screen ke Dashboard atau halaman Login.
2. **e_wallet_provider.dart**: Modul raksasa (12KB) yang mengatur alur kompleks dari penambahan *wallet* eksternal hingga kalkulasi saldo kumulatif antar akun.
3. **transaction_provider.dart**: Menarik data (`fetch`) riwayat transaksi, menghitung ulang total saldo (`income - expense`), dan menerapkan filter bulanan/kategori untuk grafik.
4. **ai_insight_provider.dart**: Mengelola logika *loading state* saat AI Gemini sedang berpikir/merespons input teks pengguna (Chatbot).
5. **settings_provider.dart**: Mengambil pengaturan lokal via `SharedPreferences` seperti preferensi Dark Mode.
6. **notification_provider.dart**: Mengontrol pembacaan notifikasi (Read/Unread).

4. Bedah Lapisan Antarmuka (UI Layer)

Folder `lib/ui/` dipisah menjadi `pages` dan `widgets`. Semua desain mematuhi aturan estetika *Glassmorphism* yang diatur secara global di `core/theme/app_theme.dart`.

A. Pages / Layar Utama (lib/ui/pages/)

- **splash/**: Pintu gerbang aplikasi, memverifikasi sesi login secara asinkron.
- **auth/**: Terdiri dari `login_page.dart` dan `register_page.dart`.
- **main/**: Memuat `main_page.dart` yang memegang logika Bottom Navigation Bar. Layar ini adalah fondasi yang memuat tab Dashboard, Transaksi, dan Profil tanpa mematikan state layar lain.
- **dashboard/**: Pusat ringkasan kas (Card Saldo) dan grafik kecil (`fl_chart`).
- **transaction/**: Memiliki banyak rute, mulai dari Input Manual (`add_transaction_page.dart`), input suara, hingga OCR Camera (`scan_receipt_page.dart`).
- **ai_advisor/**: Halaman interaktif Chat AI mirip ChatGPT untuk konsultasi bisnis.
- **portfolio/ & budget/**: Halaman diagram mendetail dan manajemen limit dana per bulan.
- **e_wallet/**: Layar manajemen sinkronisasi dompet digital.
- **settings/**: Mengatur profil, cetak laporan (Report Generator PDF), dan izin perangkat (Microphone & Kamera).

B. Widgets Kustom (`lib/ui/widgets/`)

- `custom_text_field.dart`: Didesain khusus agar seragam di semua form (memiliki efek transisi fokus).
- `transaction_card.dart`: Komponen kartu list riwayat transaksi.
- `custom_popup.dart`: Menggantikan `AlertDialog` bawaan Flutter dengan desain yang lebih premium dan elegan.

5. Konfigurasi Sistem Utama (`lib/main.dart` & `app.dart`)

- **main.dart**: Titik masuk eksekusi (Entry point). Memanggil `WidgetsFlutterBinding.ensureInitialized()`, menginisialisasi Firebase App (`DefaultFirebaseOptions`), membaca `.env`, mendaftarkan `get_it` Service Locator, lalu memanggil class `App`.
- **app.dart**: Merupakan root dari `MaterialApp`. Di sinilah `MultiProvider` dikonfigurasi secara hierarkis.

Kesimpulan: Arsitektur Camelio sangat matang (*production-ready*). Pemisahan MVVM memungkinkan modul AI Gemini dapat di-maintenance atau diubah di masa depan tanpa merusak tampilan UI aplikasi, menjadikan kodenya sangat solid untuk sebuah skripsi/tugas akhir sistem cerdas.